

Padrão Universal de Tratamento de Erros **n8n** Enterprise

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Data: Julho 2026

Status: Homologado

Versão: 2.0

pdi-orquestracao-corporativa-n8n-a1

1. Contexto

A operação de Automação & Infraestrutura da V4 Company mantém 10 workflows críticos em produção no n8n: 7 workflows de SDR IA (atendimento inteligente de leads) e 3 workflows de Command Center (monitoramento e orquestração da instância).

Em meados de junho de 2026, identificamos um padrão recorrente: **todos os 7 workflows SDR IA apresentavam falhas silenciosas**. Erros aconteciam, os workflows paravam ou degradavam, e ninguém era notificado. Algumas falhas passavam **dias sem detecção**, impactando diretamente o SLA de atendimento aos leads.

Este relatório documenta o PDI (Padrão Universal de Tratamento de Erros) implementado para resolver esse problema de raiz: não apenas corrigindo as falhas pontuais, mas estabelecendo um **padrão enterprise reutilizável** para todos os workflows da operação.

2. Diagnóstico

2.1 Workflows Afetados

WORKFLOW	ERRO	IMPACTO
ADPLAN	JS timeout 25min: event loop bloqueado por SplitInBatches ausente	P0
SIGNOR	Task runner desconectado: servidor precisou de tuning	P0
Genics	Redis Cloud inacessível: DNS externo sem fallback	P0
SOFIA	Rate limit Chatwoot: sem wait entre requisições	P1
PRO ANÁLISES	<code>toDateTime undefined</code> : campo opcional sem validação	P1
Schwalm	Null constraint em telefone: insert no Supabase sem validação	P1
V4 INTERNO	404 no Ekyte: board ID mudou sem atualizar o workflow	P1

P0 = Bloqueia operação · P1 = Degrada experiência

2.2 Command Centers

Os 3 workflows de infraestrutura também operavam sem qualquer tratamento de erro:

WORKFLOW	TRIGGER	RISCO
[CC] Collector	A cada 2 min	Falha silencia → dashboard desatualizado
[CC] Heartbeat	A cada 5 min	Falha silencia → falso positivo de saúde
[CC] Metrics	A cada 1 h	Falha silencia → lacuna histórica

2.3 Causa Raiz

Ausência de padronização: cada workflow tratava erro do seu jeito improvisado: ou simplesmente não tratava. Não havia taxonomia de erros, política de retry, notificação, persistência ou proteção contra cascata. O resultado era um ecossistema frágil onde uma falha em um nó poderia derrubar todo o fluxo sem deixar rastro.

3. Arquitetura da Solução

Três camadas obrigatórias e complementares. Não são alternativas: um workflow enterprise precisa das três.

CAMADA 1

Node-Level · Auto-cura local

`retryOnFail` + `continueOnErrorOutput` em cada nó falível.
Captura ~73% das falhas transientes (429, 503, timeout) no próprio nó: sem overhead de workflow externo.

~73%

CAMADA 2

Error Handler Central · Rede de segurança

Workflow único que captura tudo que escapar da Camada 1. Classifica severidade por taxonomia, notifica Slack em < 1 min, persiste na Dead Letter Queue. 10 nós, validado com n8nac.

10 nós

CAMADA 3

Circuit Breaker + Dead Letter Queue · Proteção sistêmica

Circuit breaker evita sobrecarga em APIs fragilizadas (abre após 5 falhas consecutivas, cooldown de 5 min, recovery automático). DLQ preserva payload completo para replay e auditoria.

2 wf

O que a Camada 1 não cura, a 2 classifica. O que a 2 classifica, a 3 protege e preserva.

3.1 Racional de Design

Por que não um error handler por workflow? Duplicação de lógica. Manter N workflows de erro idênticos é custoso e propenso a dessincronia. Um handler centralizado reduz a superfície de manutenção a um único ponto.

Por que 3 camadas e não 2? A Camada 1 (node-level) é a mais barata e captura a maioria dos casos: mas não cobre erros de lógica (dado inválido, runtime). A Camada 2 classifica o que escapar, mas não protege APIs contra cascata. A Camada 3 existe exatamente para isso: impedir que uma API falha receba golpes em sequência.

E se o Error Handler Central cair? Os erros ficam no log interno do n8n até o handler voltar. Há perda de notificação, mas não de dado: a DLQ é populada no Supabase, que tem SLA próprio. Além disso, o Circuit Breaker Monitor roda independentemente a cada 5 min.

4. Taxonomia de Erros

Todo erro é classificado no momento da captura em exatamente uma das 8 classes abaixo. Sem ambigüidade, sem exceção.

CLASSE	SEVERIDADE	RETENTÁVEL?	EXEMPLO	AÇÃO
server_error	CRITICAL	Sim (3x)	502, 503	Retry + DLQ + Slack #incidents
rate_limit	WARNING	Sim (c/ backoff)	429	Backoff + jitter + DLQ
timeout	CRITICAL	Sim (3x)	ETIMEDOUT	Retry + DLQ + Slack
client_error	WARNING	Nunca	400, 401, 403	DLQ + notificar equipe
data_validation	WARNING	Nunca	null constraint	DLQ + notificar
network_dns	CRITICAL	Sim (3x)	ENOTFOUND	Retry + DLQ + Slack
runtime	CRITICAL	Nunca	Task runner off	Alarmar imediatamente
resource_exhaustion	CRITICAL	Nunca	OOM	Alarmar imediatamente

5. Envelope Padrão de Erro

Todo workflow que captura um erro DEVE produzir este envelope JSON. O `correlationId` é estável entre retries: permite rastrear uma falha desde a primeira tentativa até o descarte na DLQ.

```
{
  "severity": "critical",
  "errorClass": "server_error",
  "correlationId": "abc123-67890",
  "workflowName": "CRM Sync",
  "workflowId": "abc123",
  "failedNode": "HTTP Request: Attio",
  "errorMessage": "Request failed with status code 502",
  "executionUrl": "https://n8n.fvmarketing.com.br/workflow/abc123/executions/67890",
  "timestamp": "2026-07-08T10:30:00.000Z"
}
```

5.1 Código de Classificação

O Code node do Error Handler Central implementa a taxonomia via regex. Abaixo, o core da lógica:

```
// Error Handler Central: Parse + Classify
const msg = (execution.error?.message || '').toLowerCase();

let errorClass = 'unknown';
let severity = 'info';

if (/5\d{2}|internal server error|upstream/.test(msg)) {
  errorClass = 'server_error'; severity = 'critical';
} else if (/429|rate limit|too many/.test(msg)) {
  errorClass = 'rate_limit'; severity = 'warning';
} else if (/timeout|etimedout|econnrefused/.test(msg)) {
  errorClass = 'timeout'; severity = 'critical';
} else if (/undefined|cannot read|null|not-null/.test(msg)) {
  errorClass = 'data_validation'; severity = 'warning';
} else if (/task runner|disconnect/.test(msg)) {
  errorClass = 'runtime'; severity = 'critical';
}

return [{
  json: {
    correlationId: [workflow.id, execution.id].join('-'),
    severity, errorClass,
    workflowName: workflow.name,
    failedNode: execution.lastNodeExecuted,
    errorMessage: (error.message || '').substring(0, 500),
    executionId: execution.id,
    timestamp: new Date().toISOString()
  }
}];
```

6. Retry Matrix

Cada tipo de nó tem sua própria política de retry. Backoff exponencial com jitter para 429 e 5xx.

TIPO DE NÓ	RETRYONFAIL	MAXTRIES	WAITBETWEENTRIES	OBSERVAÇÃO
HTTP Request	✓	3	5000 ms	Sempre configurar
Supabase	✓	2	3000 ms	Falhas de rede raras
Slack	✓	3	5000 ms	Rate limit 429 frequente
WhatsApp / API externa	✓	3	5000 ms	Instabilidade comum

TIPO DE NÓ	RETRYONFAIL	MAXTRIES	WAITBETWEENTRIES	OBSERVAÇÃO
Google Sheets / Drive	✓	2	5000 ms	429 em pico
Redis	✓	2	2000 ms	Conexão local
Postgres	✓	2	3000 ms	Timeout de query
n8n API	✓	3	5000 ms	Rate limit interno
Code node	✗	-	-	NUNCA retentar: erro de lógica

SEMPRE RETENTAR

- 5xx (server_error)
- 429 (rate_limit)
- 408/409/425 (timeout)
- ETIMEDOUT / ENOTFOUND

NUNCA RETENTAR

- 4xx (exceto 408/409/425/429)
- Data validation
- Runtime / OOM
- 401/403 (credencial expirou: alertar)

CONFIGURAÇÃO

```
{
  retryOnFail:
    true,
  maxTries: 3,
  waitBetweenTries:
    5000
}
```

6.1 Backoff com Jitter

Para falhas persistentes (429, 5xx), usar loop customizado com backoff exponencial:

```
// Fórmula: min(maxDelay, baseDelay * 2^(attempt-1)) ± 25%
waitSeconds = Math.min(300, 5 * Math.pow(2, attempt - 1)) * (0.75 + Math.random() * 0.5);
```

7. Error Handler Central

O workflow central que orquestra todo o tratamento de erro. Recebe erros de todos os workflows, classifica, notifica e persiste.

Pipeline de Execução

Regras de Severidade

Error Trigger (de qualquer workflow)

```
→ Code: Parse & Classify
  → IF: Severity Critical?
    └─ critical
      |   → Slack #incidents
      |   → Supabase INSERT DLQ
      |   → HTTP PATCH Circuit Breaker
    └─ warning/info
      |   → Slack #alerts
      |   → Supabase INSERT DLQ
```

CONDIÇÃO

SEVERIDADE

5xx, timeout, DNS, OOM,
runtime

CRITICAL

429, 4xx, data_validation

WARNING

Não classificado

INFO

Workflow
"pagamento/auth/billing"

CRITICAL
(override)

Métricas do Workflow

10

NÓS

3

CANAIS DE SAÍDA



VALIDADO N8NAC

8. Circuit Breaker & Dead Letter Queue

8.1 Circuit Breaker

Protege APIs contra cascata de falhas. O estado é **isolado por workflow**.

PARÂMETRO	VALOR
FAILURE_THRESHOLD	5 falhas consecutivas
COOLDOWN_MS	300.000 (5 min)
Recovery	Automático via PATCH n8n API
Monitor	A cada 5 min

8.2 Dead Letter Queue

Persistência **permanente** de todas as falhas não-recuperáveis. Diferente do prune de execuções do n8n (que apaga dados antigos), a DLQ preserva para auditoria.

CAMPO	TIPO	DESCRIÇÃO
correlation_id	TEXT	Estável entre retries
error_class	TEXT	Taxonomia (CHECK)
severity	TEXT	critical/warning/info
payload	JSONB	Dado completo do erro

```
closed --(5 falhas)→ open --(5 min)→ half-open --(sucesso)→ closed
```

CAMPO	TIPO	DESCRIÇÃO
status	TEXT	pending → investigating → resolved

Views de Monitoramento

- vw_error_health_score**: Score geral de saúde (0-1)
- vw_circuits_open_now**: Circuitos abertos neste momento
- vw_error_summary_24h**: Top erros das últimas 24h
- vw_retry_success_rate**: Taxa de auto-cura por nó

8.3 Schema Supabase (v2.1)

Foram adicionadas 4 tabelas ao schema existente: `error_dlq` , `error_circuit_breaker` , `error_retry_log` e `error_alert_config` , com índices de performance e 4 views de monitoramento. Schema completo em `PDI/3-supabase/supabase-schema-v2.1.sql` .

9. Plano de Retrofit

5 workflows SDR IA já tiveram correções aplicadas nos JSONs locais em 22/06/2026. Estão pendentes de push para o n8n. O retrofit completo está estimado em **4 dias**.

WORKFLOW	ERRO	CORREÇÃO	ESFORÇO
ADPLAN	JS timeout 25min	SplitInBatches + Code otimizado	20 min
SOFIA	Rate limit Chatwoot	Wait 500ms + Error Workflow	10 min
PRO ANÁLISES	toDateTime undefined	try/catch toDateTime	10 min
Schwalm	Null constraint telefone	Validação + IF	10 min
V4 INTERNO	404 Ekyte board ID	Board ID map + IF	10 min

5WORKFLOWS
CORRIGIDOS**~1h**ESFORÇO DE
CORREÇÃO**10**WORKFLOWS P/
ERROR HANDLING**4 dias**RETROFIT
COMPLETO

9.1 Fases de Execução

FASE	DURAÇÃO	ATIVIDADES
1. Push + Schema	1 dia	Rodar migration no Supabase · Push 5 workflows · Validar com n8nac
2. Configurar error handling	2 dias	Vincular Error Handler nos 10 workflows · Configurar retryOnFail · Publicar
3. Validação	1 dia	Rodar cada workflow · Falha provocada · Confirmar DLQ + Slack + recovery

10. Anti-Patterns

Erros reais observados nos workflows SDR IA que o padrão veio eliminar.

ONERROR SEM MAIN[1]

Erro descartado silenciosamente. O nó falha, mas ninguém vê. Sempre usar `onError:` `"continueErrorOutput"` + segundo output conectado ao handler.

ERROR WORKFLOW NO MESMO CANAL

Se o workflow de erro usa o mesmo Slack que o principal, uma falha no Slack cria recursão infinita. Usar canais separados.

RETENTAR 4XX

Queima API credits à toa. Erro do cliente (400, 401, 403, 404) não vai mudar na próxima tentativa. Só retentar 5xx / timeout / DNS.

NÃO PUBLICAR O ERROR HANDLER

Código antigo rodando mesmo depois de "corrigido". Sempre publish (Shift+P) após configurar ou alterar.

200 NO ERRO

PAYLOAD SEM CORRELATIONID

Sem um ID estável entre retries, não é possível ligar uma falha à sua causa original. O `correlationId` é obrigatório.

Se a resposta é 200 mas houve falha interna, quem chamou nunca sabe. Propagar status HTTP correto.

11. Métricas de Sucesso



11.1 Consultas de Acompanhamento

```
-- Health score geral
SELECT * FROM vw_error_health_score;

-- Circuitos abertos agora
SELECT * FROM vw_circuits_open_now;

-- Top 10 workflows com erro (7 dias)
SELECT workflow_name, COUNT(*) AS total
FROM error_dlq
WHERE created_at > now() - INTERVAL '7 days'
GROUP BY workflow_name
ORDER BY total DESC
LIMIT 10;

-- Erros não revisados
SELECT * FROM error_dlq
WHERE status = 'pending_review'
ORDER BY severity DESC, created_at DESC
LIMIT 50;
```

12. Entregas do PDI

PASTA	CONTEÚDO	FORMATO
1-standards/	Documentação do padrão, taxonomia, envelope de erro, retry matrix (609 linhas)	Markdown
2-workflows/	Error Handler Central + Circuit Breaker Monitor, validados com n8nac	.workflow.ts
3-supabase/	Schema v2.1 (4 tabelas + 4 views) + guia de migração	SQL + Markdown
4-retrofit/	Planos de retrofit para SDR IA (7 workflows) e Command Center (3 workflows)	Markdown
5-monitoring/	Consultas SQL de dashboard e regras de alerta	Markdown
6-automation/	Scripts de deploy automatizado (validação + push + migração)	Shell
7-apresentacao/	Deck interativo HTML (14 slides) + script de demonstração + este relatório	HTML + Markdown

12.1 Comandos de Deploy

```
# 1. Migração do schema Supabase
bash PDI/6-automation/run-migration.sh

# 2. Validar workflows de orquestração
npx n8nac skills validate "PDI/2-workflows/Error Handler Central.workflow.ts"
npx n8nac skills validate "PDI/2-workflows/Circuit Breaker Monitor.workflow.ts"

# 3. Push para o n8n
npx n8nac push "PDI/2-workflows/Error Handler Central.workflow.ts" --verify

# 4. Deploy completo (automatizado)
bash PDI/6-automation/deploy-error-handler.sh
```

13. Decisões e tradeoffs

Escolhas derivadas do conteúdo real da atividade, com custo e benefício explícitos.

- **Handler central único:** manutenção em um único ponto com 10 nos validados com n8nac. Se o handler cair, perde-se notificação temporária, mas não o dado, pois a DLQ está no Supabase e o Monitor roda a cada 5 min.
- **Três camadas obrigatórias:** Camada 1 captura cerca de 73% das falhas transientes sem overhead externo, Camada 2 classifica e notifica em menos de 1 min e Camada 3 impede cascata. Custo de mais configuração por no em troca de auto-cura acima de 70%.
- **Code node nunca retenta:** erro de lógica como `toDateTime undefined` ou `null constraint` em telefone não se resolve com `retry`. Exige validação com IF e `try/catch` para não queimar créditos de API.
- **Breaker isolado por workflow:** limiar de 5 falhas consecutivas e cooldown de 5 min com recovery automático. Pausa temporária em troca de estabilidade sistêmica.
- **DLQ permanente no Supabase:** schema v2.1 com 4 tabelas e 4 views preserva payload com `correlationId` estável para replay. Custo de manter migração e views como `vw_error_health_score` e `vw_circuits_open_now`.

14. Impacto no negócio

Com 7 workflows SDR IA sem notificação e detecção levando dias, cada falha silenciosa derrubava o SLA de atendimento a leads sem ninguém saber. O padrão reduz a detecção para menos de 1 min via Slack, eleva a auto-cura para mais de 70% com `retry` no nó e zera circuitos abertos sem alerta (de 100% para 0%). O retrofit completo cabe em 4 dias, com cerca de 1h para as 5 correções pontuais já mapeadas (20 min no ADPLAN e 10 min em cada um dos outros 4), o que corta risco operacional e retrabalho de investigação manual.

15. Referências de estudo

Material para aprofundar os temas usados nesta atividade.

- **Curso:** Automação e Orquestração n8n do Básico ao Avançado, na Udemy.
- **Vídeo:** Error Trigger in n8n, tratamento centralizado de erros, no YouTube, canal oficial n8n.
- **Doc:** n8n Docs, Error handling with Error Trigger workflows, na plataforma n8n Docs.
- **Doc:** Supabase Docs, Database tables, views and indexes, na plataforma Supabase Docs.

PDI: Padrão Universal de Tratamento de Erros n8n Enterprise

Autor: Marcos Luciano · Tech Lead · FV Marketing / V4 Company

Data: Julho 2026 · Versão 2.0 · Status: Homologado

Slug: [pdi-orquestracao-corporativa-n8n-a1](#)

Este documento é parte integrante da atividade de PDI técnico.